29

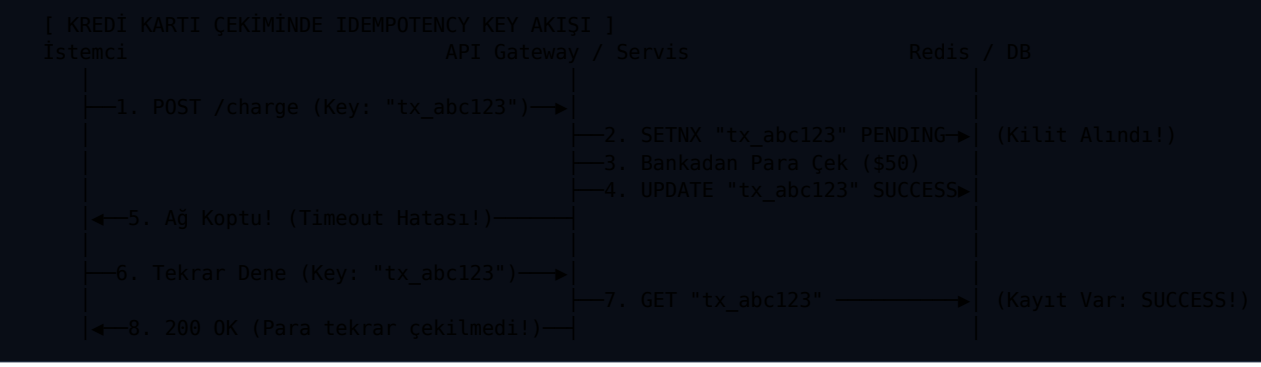
BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Idempotency (Tekrarlanabilirlik): Ağ Yeniden Denemelerinde Müker-rerliği Önleme

Idempotency Patterns: Preventing Double Charges & Duplicates

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Asansör çağrı düğmesini düşünün: Düğmeye bir kez basarsanız ışığı yanar ve asansör kata doğru hareket eder. Sabırsızlanıp düğmeye 10 kez daha basarsanız ne olur? Asansör 10 kat daha hızlı gelmez, 10 tane ayrı asansör de çağrılmaz! Düğmeye 1 kez basmakla 100 kez basmak aynı sonucu üretir. İşte matematikte ve yazılımda buna `Idempotent` (Tekrarlanabilir) denir: Kullanıcı interneti koptu diye 'Öde' butonuna 5 kez tıklasa bile kredi kartından sadece 1 kez 50 TL çekil-melidir!

🇬🇧 ENGLISH (ELI5)

Think of an elevator call button: Pressing it once illuminates the light and dispatches the cab. If you impatiently mash the button 10 more times, does it arrive 10 times faster? Does it spawn 10 separate elevators? No! Pressing it once or 100 times yields the exact same state. That is `Idempotency`. If a customer's phone switches from Wi-Fi to 5G during checkout and re-submits a payment 5 times, their credit card must be charged exactly once!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık sistemlerde ağ zaman aşımları (Timeout) nedeniyle istemciler istekleri yeniden dener (Retry). Bir POST isteği doğası gereği idempotent değildir. Çözüm: 1) İstemci Tarafında Benzersiz Anahtar (Idempotency Key): İstemci isteği göndermeden önce bir UUID üretir (`Idempotency-Key: 9b1deb4d...`) ve HTTP başlığına ekler. 2) Dağıtık Kilit & Durum Tablosu (Redis / DB): API Gateway anahtarı alır; `SET key PENDING NX EX 120` ile atomik kilit oluşturur. İşlem tamamlandığında sonuç (Response Body) bu anahtarın altına kaydedilir. 3) Mükerrer İstek Geldiğinde: Eğer anahtar `SUCCESS` durumundaysa, işlem tekrar çalıştırılmaz; önbellekteki ilk başarılı yanıt aynen geri dönülür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

In distributed microservices, network timeouts inevitably trigger client retries. While GET and PUT are inherently idempotent, HTTP POST mutations are not. Enterprise Idempotency Pattern: 1) Client Idempotency Key: The client generates a unique UUID (`Idempotency-Key: 9b1deb4d...`) passed in headers. 2) Atomic State Locking: The receiver executes an atomic lock (`SET key PENDING NX EX 60`) in Redis or an RDBMS unique table constraint. 3) Response Caching: When the transactional operation succeeds, the final response payload is persisted against the idempotency key. Subsequent retry attempts detect the existing key and return the cached receipt immediately without re-executing side effects.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Idempotency Key Motoru (Ödeme Güvenlik Kalkanı)
import java.util.concurrent.*;

public class PaymentIdempotencyEngine {
    public enum TxStatus { PENDING, COMPLETED }
    public record TxRecord(TxStatus status, String responsePayload) {}

    private final ConcurrentMap<String, TxRecord> store = new ConcurrentHashMap<>();

    public synchronized String processPayment(String idempotencyKey, double amount) {
        TxRecord existing = store.get(idempotencyKey);
        if (existing != null) {
            if (existing.status() == TxStatus.COMPLETED) {
                IO.println("⚡ Önbellekteki Makbuz Döndü: " + existing.responsePayload());
                return existing.responsePayload();
            }
            throw new IllegalStateException("İşlem şu anda başka bir thread tarafından yürütülüyor!");
        }

        // 1. Durumu PENDING yap
        store.put(idempotencyKey, new TxRecord(TxStatus.PENDING, null));

        // 2. Banka POS simülasyonu
        IO.println(">>> Bankadan $" + amount + " çekildi! (Kart İşlemi Yapıldı)");
        String receipt = "RECEIPT_TX_" + System.currentTimeMillis();

        // 3. Durumu COMPLETED yap ve sonucu sakla
        store.put(idempotencyKey, new TxRecord(TxStatus.COMPLETED, receipt));
        return receipt;
    }
}

void main() {
    var engine = new PaymentIdempotencyEngine();
    String key = "req_uuid_550e8400";

    // 1. İstek (Başarılı)
    engine.processPayment(key, 100.0);
    // 2. Ağ hatası nedeniyle aynı isteğin tekrarı (Tekrar para çekilmez!)
    engine.processPayment(key, 100.0);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Idempotency anahtarlarını veritabanında sonsuza kadar saklamayın! Bu tablolar milyonlarca gereksiz anahtarla şişer. Idempotency anahtarlarına 24 veya 48 saatlik bir TTL (Time-To-Live) belirleyin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not persist idempotency keys indefinitely. Storing transient keys permanently bloats storage. Enforce a rolling 24-to-48 hour TTL window, which easily exceeds maximum client retry horizons.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Idempotency Key, ağ hatalarında mükerrer para çekilmesini ve kayıt açılmasını engeller.

• HTTP POST istekleri istemcinin ürettiği benzersiz bir token ile idempotent kılınır.

• Stripe, PayPal ve Adyen gibi ödeme devlerinin tüm API'leri bu desenle korunur.

🎯 Key Takeaways for the Interview

• Idempotency keys eliminate double-billing and duplicate resource creation during retries.

• Pairing client-generated UUID tokens with atomic server-side state locks ensures idempotency.

• Industry payment APIs (Stripe, Adyen) mandate Idempotency-Key headers on all mutations.